

Ticket Data Structure — Design

Team 27 · Grand Prix Ticketing Redesign · Sprint 1, Week 2
Blake Leonard (Developer 1) · Card T8 · 4 hours

1. What this is

What the app needs to store for the ticket explorer feature, decided before any code is written. Kept deliberately small — one collection, read-only from the app. Sprint 2 builds against this.

2. One collection: ticketType

Everything the browse, detail and compare screens need sits in a single collection. No relationships, no joins, no user-owned data.

Field	Type	Required	What it holds
id	string	yes	Firestore document id
name	string	yes	e.g. "Grandstand — Turn 3, 4 day"
daysCovered	array of strings	yes	e.g. ["Thu", "Fri", "Sat", "Sun"]
zone	string	yes	Where at the circuit, e.g. "Turn 3"
priceAud	number	yes	Price in AUD
viewDescription	string	no	One line on what you can see from there
includes	array of strings	no	What the ticket covers
excludes	array of strings	no	What it does not cover — the useful bit
schemaVersion	number	yes	Always 1, per the boilerplate convention

3. Security rules

Simplest option that is still safe:

- Any signed-in user can read ticketType.
- Nobody can write to it from the app. It is reference data — we seed it ourselves from the Firebase console.
- Everything else stays denied by default, as the boilerplate rules already do.

This means no write rules to get wrong, and nothing a user can corrupt.

4. Deliberately not doing yet

- No saved shortlists or favourites — needs a second collection and user-owned data. Sprint 3 if we want it.
- No seat-level data. Zone is enough for the compare screens.
- No images. Cloud Storage needs the paid plan, so view descriptions are text for now.
- No admin screens. Sample data goes in by hand.

5. Open questions

- Does the client have real ticket names and prices we can use, or do we make up representative samples?
- How many ticket types are we modelling — five or six is plenty to prove the feature.